Universidad San Carlos de Guatemala

Facultad de Ingenieria

Informe de Desarrollo

Giovanni Josue Villegas Alvarez

202501086

Contents

Introducción 3
Descripción de la Implementación 4
Estructura del sistema 4
Manejo de datos 4
Manejo de archivos 5
Funcionalidades implementadas 5
Problemas Encontrados Durante el Desarrollo 6
Manejo de arreglos sin ArrayList 6
Validaciones de reglas de negocio 6
Manejo de archivos 6
Conexión entre menús 6
Actualización de tablas (interfaz) 7
Soluciones Adoptadas 7
Manejo de arreglos 7
Validaciones 7
Manejo de archivos 7
Navegación entre menús 8
Actualización de interfaz 8
Resultados 8
Mejoras Futuras 8
Conclusión 9

Introducción

El presente informe describe el desarrollo del sistema BiblioSystem, el cual consiste en una aplicación en Java diseñada para gestionar el funcionamiento de una biblioteca universitaria. El sistema permite administrar libros, usuarios y préstamos, cumpliendo con las reglas de negocio establecidas en el enunciado del proyecto.

El objetivo principal del proyecto fue aplicar los conocimientos adquiridos en el curso de Introducción a la Programación, especialmente en el uso de estructuras de datos, manejo de archivos, validaciones y desarrollo de lógica computacional.

Descripción de la Implementación

El sistema fue desarrollado utilizando el lenguaje de programación Java, siguiendo una estructura basada en el patrón **Modelo - Vista - Controlador (MVC)** para mantener organizado el código.

Estructura del sistema

El sistema se divide en tres partes principales:

- **Modelo:** Contiene las clases que representan los datos del sistema, como:
    - Usuario
    - Libro
    - Préstamo
- **Vista:** Contiene las interfaces gráficas (menús), como:
    - Login
    - Menú Administrador
    - Menú Operador
    - Menú Estudiante
- **Controlador:** Se encarga de manejar la lógica del sistema, validaciones y operaciones, principalmente en la clase:
    - ControlBiblioteca

Manejo de datos

Debido a las restricciones del proyecto, no se utilizaron estructuras como ArrayList, por lo que:

- Se utilizaron **arreglos estáticos** para almacenar:
    - Libros
    - Usuarios
    - Préstamos
- Se implementaron recorridos manuales con ciclos for para:

    - Buscar elementos
    - Validar duplicados
    - Actualizar información

Manejo de archivos

El sistema utiliza archivos de texto para guardar información:

- **cuentas.txt**
    - Guarda usuarios (operadores y estudiantes)
- **prestamos.txt**
    - Guarda los préstamos realizados

Estos archivos se leen al iniciar el sistema y se actualizan cuando se realizan operaciones importantes, tal como lo indica el enunciado

Funcionalidades implementadas

El sistema cumple con los módulos principales:

- Autenticación de usuarios
- Gestión de libros (crear, modificar, eliminar, buscar)
- Gestión de usuarios
- Control de préstamos y devoluciones
- Validación de reglas de negocio
- Generación de reportes en HTML
- Registro de operaciones en bitácora

Además, se implementaron validaciones importantes como:

- Límite de 3 préstamos por estudiante
- No permitir eliminar libros con préstamos activos
- Validación del ISBN (10 a 13 dígitos)

- Control de préstamos vencidos

Problemas Encontrados Durante el Desarrollo

Durante el desarrollo del proyecto se presentaron varios problemas:

Manejo de arreglos sin ArrayList

Fue complicado trabajar únicamente con arreglos estáticos, ya que:

- No se pueden redimensionar fácilmente
- Se debe controlar manualmente la posición de los datos

Validaciones de reglas de negocio

Implementar todas las reglas del sistema fue complejo, especialmente:

- Validar préstamos activos
- Controlar préstamos vencidos
- Evitar eliminación de registros con dependencias

Manejo de archivos

Se presentaron dificultades en:

- Lectura correcta de archivos
- Formato de datos (separadores)
- Evitar errores al guardar información

Conexión entre menús

Uno de los problemas principales fue conectar correctamente:

- Menú Administrador
- Menú Operador

- Menú Estudiante

Ya que cada uno debía tener permisos diferentes.

Actualización de tablas (interfaz)

Se detectó que algunas tablas no se actualizaban automáticamente después de realizar acciones como:

- Registrar préstamos
- Eliminar libros

Soluciones Adoptadas

Para resolver los problemas anteriores, se implementaron las siguientes soluciones:

Manejo de arreglos

- Se utilizó una variable contador para controlar la cantidad de elementos
- Se implementaron métodos para buscar posiciones dentro del arreglo

Validaciones

- Se centralizaron las validaciones dentro del controlador
- Se crearon métodos específicos para cada regla del negocio

Manejo de archivos

- Se utilizó lectura línea por línea
- Se definió un formato consistente usando separadores (por ejemplo ;)
- Se validó la información antes de guardarla

Navegación entre menús

- Se pasó como parámetro el sistema principal entre ventanas
- Se controló el acceso según el rol del usuario

Actualización de interfaz

- Se agregaron métodos para refrescar tablas después de cada operación
- Se recargaron los datos desde el controlador

Resultados

El sistema desarrollado cumple con los requerimientos del proyecto:

- Permite gestionar correctamente libros y usuarios
- Controla los préstamos y devoluciones
- Aplica todas las reglas de negocio
- Genera reportes en HTML
- Mantiene persistencia de datos mediante archivos

Además, el sistema presenta una estructura organizada y funcional.

Mejoras Futuras

Algunas mejoras que podrían implementarse en el futuro son:

- Uso de base de datos en lugar de archivos de texto
- Mejora de la interfaz gráfica
- Implementación de búsquedas más eficientes
- Manejo dinámico de estructuras de datos (ArrayList)
- Sistema de notificaciones para préstamos vencidos

Conclusión

El desarrollo de este proyecto permitió reforzar conocimientos fundamentales de programación en Java, especialmente en el manejo de estructuras de datos, validaciones y lógica del sistema.

A pesar de las dificultades presentadas, se logró implementar un sistema funcional que cumple con los objetivos planteados, demostrando la importancia de una buena organización del código y del análisis previo del problema.